

Social Media App - Project Documentation

Social Media App - Complete Project Architecture

This document provides a comprehensive breakdown of your project's directory structure. It explains the purpose of each folder, what kind of code lives inside it, and provides examples so you can easily understand your codebase.

1. Root Directory (Main Project Folder)

When you open `e:\Social\_Media\_App`, you see several folders. Here is what they do:

* **`android/`, `ios/`, `windows/`, `macos/`, `linux/`, `web/`**:

These are platform-specific folders generated by Flutter. They contain the native code required to build the app for each respective platform.

* *Note: You generally don't need to touch these unless you are adding native permissions (like camera/storage in AndroidManifest.xml) or native SDKs.*

* **`ai\_models/`**:

This folder contains your Python-based Artificial Intelligence models and backend scripts (e.g., YOLO object detection, deepfake detection, `app.py`). It is completely separate from the Flutter app code.

* **`lib/`**:

This is the most important folder. **99% of your app's code is written here.**

* **`pubspec.yaml`**:

The configuration file where all external packages (like `supabase\_flutter`, `go\_router`) and assets (images/fonts) are registered.

2. Inside the `lib/` Folder

The `lib` folder follows a **Feature-First Architecture**. This means the code is divided by features (e.g., Home, Chat, Profile) rather than by type (e.g., all screens together, all controllers together). This is a highly professional and scalable approach.

`lib/core/`

This folder contains the foundational building blocks of your app that are shared across *multiple* features.

Folder Name	Purpose & Explanation
-------------	-----------------------

---	---
-----	-----

`constants/`	Contains app-wide constants like string values, numerical constants, or API endpoints.
------------------	--

`router/`	Manages the navigation of the app (e.g., using `go_router`). Determines how screens transition.
---------------	---

`secrets/`	Stores sensitive information like Supabase URL and Anon Keys (`supabase_config.dart`).
----------------	--

`services/`	Contains services that communicate with external APIs or databases (e.g., `supabase_service.dart`).
`theme/`	Defines the visual look of the app (colors, fonts, light/dark mode) (`app_theme.dart`).
`utils/`	Helper functions that can be used anywhere (e.g., date formatting, validation logic).
`widgets/`	Reusable UI components used across different features (e.g., a custom `PrimaryButton`, `CustomTextField`).

Example of code inside `lib/core/services/supabase_service.dart`:

```
dart
class SupabaseService {
  final SupabaseClient client = Supabase.instance.client;

  // Example function to fetch posts
  Future<List<Map<String, dynamic>>> getPosts() async {
    return await client.from('posts').select().order('created_at', ascending: false);
  }
}
```

lib/features/

This is where the actual functionality of your app lives. Each feature has its own self-contained folder.

1. `auth/` (Authentication)

Handles User Login, Signup, and Password recovery.

Code: UI for Login Screen, Sign Up Screen, and the logic to authenticate users via Supabase.

2. `home/` (Main Feed & Status)

The main screen of your app.

Code: Contains the `PostCard` (UI for a single post), `StatusList` (Stories/Status UI), and the Feed screen.

Example: Here is where we recently fixed the Share and Status features!

3. `chat/` (Messaging)

Handles real-time messaging between users.

Code: Chat list screen, Individual chat room UI, and real-time database listeners for new messages.

4. `profile/` (User Profile)

Displays user details, their posts, and shared posts.

Code: `ProfileScreen`, logic to fetch a user's specific posts, edit profile options.

5. `dashboard/`

The main wrapper around the app that usually contains the **Bottom Navigation Bar**. It connects Home,

Search, Post Creation, etc.

6. `post\_creation/`

Where users create new posts or upload images.

* **Code:** Camera/Gallery picker UI, logic to upload image to Supabase Storage, and save post details to the database.

7. `notifications/`

Handles in-app notifications (likes, comments, new followers).

8. `search/`

Allows users to search for other profiles or posts.

9. `settings/`

App preferences, logout button, and theme toggling.

10. `splash/`

The very first loading screen when the app opens. It checks if the user is already logged in or not, and redirects them to either `auth` or `dashboard`.

Example of code inside a feature (`lib/features/home/screens/home\_screen.dart`):

```
``dart
class HomeScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Feed')),
      body: Column(
        children: [
          StatusList(), // Shows stories at the top
          Expanded(
            child: PostFeed(), // Shows scrolling list of posts
          ),
        ],
      ),
    );
  }
}
```

Summary

Your app is structured extremely well. If a new developer joins the project and you tell them "Fix a bug on the

Profile screen", they will immediately know to look in ``lib/features/profile/``. If you tell them "Change the primary color of the app", they will instantly look in ``lib/core/theme/``.

> [!TIP]

> **About PDF Export:**

> Since you are viewing this in the Gemini interface, this entire document is formatted beautifully. If you want this specifically as a PDF file, you can simply **Press Ctrl+P (or Cmd+P) on this screen -> Destination: Save as PDF -> Save**.

> This will generate a perfectly formatted PDF of this entire architecture guide for your personal records!